

Dimensionality reduction and unsupervised learning

Dataset: Olivetti faces, 400 photographs

How to use these notes

The primary text for this lecture is Géron, Chapters 7–8. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

Contents

1	The first lecture with no target column	2
1.1	The brief	2
1.2	Before anything is fitted: can this work at all?	3
2	Derivation: PCA via the SVD	3
2.1	Choosing d without guessing	5
3	Reducing the dimension	5
3.1	One procedural condition, because it is invisible	5
4	Clustering, and how to choose k	5
4.1	k -means, in five lines	6
4.2	Why inertia cannot choose k	6
4.3	Before reading any silhouette: what does nothing score?	7
4.4	The mean hides the shape	7

5	Putting the two halves together	8
6	Beyond k-means	8
6.1	DBSCAN	8
6.2	Gaussian mixtures	8
6.3	Anomaly detection, two ways	8
7	Spending the forty labels	9
7.1	The diagnosis was right, and the obvious repair still fails	9
8	The notebook	9
9	Exercises	10
	Summary	10

1 The first lecture with no target column

Seven lectures, and every one was handed the answers — most recently the cover types that Lecture 7’s ensembles were scored against: a price for every district, a digit for every image, a survivor flag for every passenger, a cover type for every plot. Today there is nothing. That changes what a metric can be, what a baseline is, and how you find out you were wrong.

Supervised	Unsupervised
You are given $(\mathbf{x}^{(i)}, y^{(i)})$	you are given $\mathbf{x}^{(i)}$ only
The metric compares prediction to truth	the metric compares the grouping to itself
The baseline is “predict the majority”	the baseline is “group at random”
You know when you are wrong	you find out when someone looks

The consequence for the whole lecture

Every number computed today is a statement about *geometry*, not about people. Whether the geometry is the people is a separate question, and it is answered by looking.

1.1 The brief

“We have a photographic archive. We do not know how many distinct people are in it. Group the photographs by person, so that a human can name a group instead of naming every photograph.” Nobody remembers who is in it, an annotator costs money per photograph, and we are funded for *forty* labels. That is the ordinary situation outside a benchmark: data is cheap, labels are not.

400 photographs of 40 people, ten each, 64×64 pixels flattened to $\mathbf{x}^{(i)} \in \mathbb{R}^{4096}$ with $x_j \in [0, 1]$ — 6.6 MB for the whole corpus. Ten times more dimensions than points.

The rule that keeps this honest

Olivetti ships with the identity of every photograph; the archive we are pretending to hold does not. So y is used in exactly two ways, both announced where they happen: as the *forty labels we paid for*, and as an *audit*, to find out what those forty could not have told us. Any other use of y is the same error as scoring on the test set, wearing different clothes.

1.2 Before anything is fitted: can this work at all?

k -means groups points close in Euclidean distance, so one measurement decides whether the rest of the lecture can work. It is an audit — it uses the hidden labels.

Euclidean distance, raw pixels	Pairs	Mean	Median
Same person	1,800	8.38	8.15
Different people	78,000	12.41	12.15

The gap is real and it is small: 6% of same-person pairs sit further apart than the *median* different-person pair.

Failure condition — stated in advance

No method that groups by Euclidean distance in raw pixels will recover the forty people cleanly. Not because the method is bad — because the representation puts a lamp and a nose on the same axis, with the same weight.

Moving the light source changes hundreds of pixel values by a large amount. Changing the person changes a few dozen by a small one. Euclidean distance adds them all up equally. Measuring this costs one cell and tells you whether you are about to tune or about to hope.

It is also not free to work there. One model-selection sweep — k -means at every k from 2 to 60, ten restarts each, scored by silhouette — costs 77 seconds for 590 runs of Lloyd’s algorithm at $O(\text{iterations} \cdot mkn)$ and 2 more for 59 silhouette scores at $O(m^2n)$. Both cost models are linear in n . Two of the three factors are properties of the problem; the third is a property of the *representation*.

2 Derivation: PCA via the SVD

The question: among all d -dimensional subspaces, which loses the least when you project onto it, and how much does it lose?

Write $\mathbf{X} \in \mathbb{R}^{m \times n}$ with one photograph per row, $\tilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^\top$ the centred data, $\mathbf{W} \in \mathbb{R}^{n \times d}$ with $\mathbf{W}^\top \mathbf{W} = \mathbf{I}_d$, and $\mathbf{P} = \mathbf{W}\mathbf{W}^\top$ the orthogonal projector, and $\|\mathbf{A}\|_F^2 = \sum_{ij} a_{ij}^2 = \text{tr}(\mathbf{A}^\top \mathbf{A})$. There is no probability in this derivation and none is needed.

Failure condition — an SVD taken without centring

Without centring, the direction of largest $\sum_i \|\mathbf{w}^\top \mathbf{x}^{(i)}\|^2$ is essentially $\bar{\mathbf{x}}$ itself, and the first component describes nothing but “this is a photograph of a face”.

Scikit-Learn’s PCA centres for you. If you write the SVD by hand it does not, and the result is a plausible-looking first component and no error message. It is the commonest silent bug in a hand-rolled PCA.

Seven steps

1. The objective. Project each centred face and measure how far it had to move:

$$E(\mathbf{W}) = \sum_i \|\tilde{\mathbf{x}}^{(i)} - \mathbf{W}\mathbf{W}^\top \tilde{\mathbf{x}}^{(i)}\|_2^2 = \|\tilde{\mathbf{X}} - \tilde{\mathbf{X}}\mathbf{W}\mathbf{W}^\top\|_F^2.$$

2. Pythagoras. $\mathbf{P}$ is an orthogonal projector, so kept and lost are orthogonal, and since $\|\tilde{\mathbf{X}}\|_F^2$ does not depend on $\mathbf{W}$,

$$E(\mathbf{W}) = \|\tilde{\mathbf{X}}\|_F^2 - \|\tilde{\mathbf{X}}\mathbf{W}\|_F^2.$$

3. As a trace. $\|\tilde{\mathbf{X}}\mathbf{W}\|_F^2 = \text{tr}(\mathbf{W}^\top \mathbf{C} \mathbf{W})$ with $\mathbf{C} = \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$, which is $n \times n$, symmetric, positive semi-definite, and $m - 1$ times the sample covariance matrix.

4. The SVD diagonalises it. With $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$,

$$\mathbf{C} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\top,$$

so the columns of $\mathbf{V}$ are the eigenvectors of $\mathbf{C}$ with eigenvalues σ_j^2 . They are the principal components.

5. Rotate the unknown. Write $\mathbf{W} = \mathbf{V}\mathbf{A}$ with $\mathbf{A}^\top \mathbf{A} = \mathbf{I}_d$; then $\text{tr}(\mathbf{W}^\top \mathbf{C} \mathbf{W}) = \sum_j \sigma_j^2 h_j$, where h_j is the squared norm of row j of $\mathbf{A}$. Both constraints are one line each: $\sum_j h_j = \|\mathbf{A}\|_F^2 = \text{tr}(\mathbf{A}^\top \mathbf{A}) = d$, and $h_j \leq 1$ because h_j is the squared length of the projection of a unit basis vector onto d orthonormal columns.

6. A weighted sum with d units to spend. The σ_j^2 are sorted, so put a full unit on each of the d largest and nothing on the rest: the maximum is $\sum_{j \leq d} \sigma_j^2$, attained at $\mathbf{W} = \mathbf{V}_d$, the first d columns of $\mathbf{V}$. Any other allocation moves weight from a larger σ_j^2 to a smaller one.

7. The error that is left. Since $\|\tilde{\mathbf{X}}\|_F^2 = \text{tr}(\mathbf{C}) = \sum_j \sigma_j^2$,

$$\min_{\mathbf{W}} E(\mathbf{W}) = \sum_{j > d} \sigma_j^2.$$

That is Eckart–Young–Mirsky in the Frobenius norm. “PCA minimises reconstruction error” is a *theorem*, not a slogan — and it names the price in advance: the tail of the spectrum, readable before you choose d .

Note also that step 2 says minimising error and maximising projected variance are *the same problem*, not two related ones.

2.1 Choosing d without guessing

By step 7 the fraction of squared error removed by keeping d components is exactly $\sum_{j \leq d} r_j$, where $r_j = \sigma_j^2 / \sum_i \sigma_i^2$. The explained variance ratio is not a heuristic; it is the theorem, normalised.

Cumulating to a target, **123 components reach 95%**. The rest of this lecture runs at 123 — a factor of 33 fewer numbers per photograph. In code that is `PCA(n_components=0.95)`: a float between 0 and 1 is read as a variance target and scikit-learn chooses d from the cumulative ratio, while an integer means “this many components” — so `n_components=1` and `n_components=1.0` do completely different things.

A principal component is a vector in $\mathbb{R}^{4096}$, so reshaping it to 64×64 makes it an image; on faces these are called **eigenfaces**, and every photograph in the corpus is the mean face plus a weighted sum of them. That is not a property of faces but of PCA: the components are directions in the input space, so whatever an input looks like, a component looks like too. And looking at the first few, they are *lighting* — exactly what the distance histogram predicted would dominate. A held-out face rebuilt from its components is recognisable well before 123 of them — identity survives a compression the pixel values do not.

3 Reducing the dimension

3.1 One procedural condition, because it is invisible

A reducer is an estimator: it is *fitted*. Fit it before the split and the held-out faces have helped choose the subspace they are then judged in. The 280 training faces, once centred, span at most a 279-dimensional subspace of $\mathbb{R}^{4096}$, so adding 120 more points genuinely changes which directions survive.

$d = 123$, 20 stratified splits	Fitted on train	Fitted on everything	Leaky wins
Reconstruction error, held out	0.00243	0.00096	20 of 20
Accuracy (logistic regression), held out	97.4%	97.4%	3 of 20

Failure condition — the decision rule, and why it is still procedural

The leak makes the reconstruction error look 61% better and moves the accuracy by nothing measurable.

Fitting an unsupervised step on everything costs you exactly when *that step's own objective is the number you report*. Reconstruction error is that objective; downstream accuracy is not — and you cannot tell which row you are in without doing the split. So put it in a Pipeline, which refits inside every fold and makes the leak structurally impossible rather than merely avoided.

4 Clustering, and how to choose k

4.1 *k*-means, in five lines

1. Choose k initial centroids; scikit-learn uses **k-means++**, which spreads them out.
2. Assign every point to its nearest centroid.
3. Move every centroid to the mean of its assigned points.
4. Repeat 2–3 until no assignment changes.
5. Repeat the whole thing n_{init} times and keep the best.

Step 5 is not optional. Lloyd’s algorithm converges — to a local minimum that depends entirely on where you started. n_{init} changed its default between scikit-learn versions, so name it.

k-means makes four assumptions whether you asked or not: the partition is a Voronoi diagram, so every cluster is *convex*; the only quantity is distance to a centre, so clusters are *spherical and equal-sized*; every point belongs to exactly one cluster, so there is no “none of these”; and k is *given*. Three of the four are false here. The first three are dropped by DBSCAN and by Gaussian mixtures, later in the lecture; the fourth, choosing k , is what the rest of this section is about.

4.2 Why inertia cannot choose k

$$J(k) = \sum_i \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_{c(i)}\|_2^2$$

is what *k*-means minimises — and it is non-increasing in k , with $J(m) = 0$ exactly. Refining a partition cannot increase the sum of squared distances to the assigned centre. *There is no interior minimum to find, at any k , on any dataset*, so “minimise inertia” selects one cluster per photograph. The elbow rule exists to work around that, and on our curve the kneedle rule — the point furthest below the chord, both axes rescaled — picks $k = 19$ where the archive has 40 — with 46% of the inertia at $k = 2$ still present at $k = 40$.

The elbow is not a criterion

It is a picture of a criterion that does not exist, and no better heuristic repairs it, because the information is not in J .

The silhouette does have an interior maximum. For a point i , with a its mean distance to its own cluster and b to the nearest other,

$$s_i = \frac{b - a}{\max(a, b)} \in [-1, 1] :$$

near +1 the point is far inside its own cluster, near 0 it is on a boundary, near −1 it is assigned to the wrong cluster. The silhouette score is the mean of s_i , penalised both for splitting a group and for merging two. It costs $O(m^2n)$ — every pair, over every feature, and both factors matter: the archive is bigger than 400, and n is larger than it needs to be.

4.3 Before reading any silhouette: what does nothing score?

Random assignment	Silhouette	ARI on the forty
$k = 10$	-0.0422 ± 0.0045	-0.0040 ± 0.0205
$k = 40$	-0.1288 ± 0.0107	-0.0089 ± 0.0244
$k = 60$	-0.1805 ± 0.0124	-0.0075 ± 0.0261

Both anchors sit at zero, and now you know the width of the noise around zero. A silhouette of 0.02 is not a discovery.

The **adjusted Rand index** compares two groupings pair by pair. With a the pairs together in both groupings and b the pairs apart in both,

$$\text{RI} = \frac{a + b}{\binom{m}{2}}, \quad \text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{1 - \mathbb{E}[\text{RI}]}.$$

RI is high for free at large k , because almost every pair is apart in both groupings. The adjusted index subtracts what chance would give, so random scores 0 in expectation at any k , perfect scores 1, and negative is worse than chance. We may compute it on the forty labelled photographs only — 780 pairs, of which just 22 are the same person — so treat it as a sanity check with a wide interval, never as a ranking.

The silhouette curve has a maximum, well above the anchor. The maximum is also low, and the curve near it nearly flat, so the chosen k is not decisively better than its neighbours.

4.4 The mean hides the shape

A silhouette diagram draws one knife per cluster, width for size and length for quality. A knife entirely left of the mean marks a cluster worse than average; knives of very unequal width mean one group has swallowed several; a knife extending into negative s_i marks points closer to another cluster than their own; and *every knife short* means the clusters overlap and the geometry has no clean gaps. Ours is the last, at every k we tried — which is the diagnosis the mean silhouette does not contain.

At $k = 40$, against the hidden identities	Value
Cleanest cluster: 10 photographs, purity	1.00
Worst cluster: 24 photographs of 10 people, purity	0.21
Mean purity across the forty clusters	0.74
Clusters holding exactly one person	16
Clusters with exactly ten members	7
Cluster sizes, smallest to largest	4 to 25

Every true group has exactly ten members. The overlap the distance histogram showed before anything was fitted is now a count — and the worst cluster is not a person at all, but a lighting condition: several different people lit and posed the same way.

5 Putting the two halves together

The clustering code is unchanged — same grid, same seed, same `n_init`. Only what k -means is looking at changed.

Both cost models lose a factor of 33 in their n : the sweep runs at 123 dimensions instead of 4,096.

6 Beyond k -means

k -means assumes convex, equal-sized clusters, total assignment and a known k . Two methods from Chapter 8 drop some of those assumptions.

6.1 DBSCAN

DBSCAN rests on three definitions. A point is a **core point** if at least `min_samples` points lie within `eps` of it. Core points within `eps` of each other are in the same cluster, and so are the non-core points in their neighbourhoods. Everything else is **noise**, labelled `-1` — so count clusters as `len(set(db.labels_) - {-1})`. DBSCAN has no `predict`: it is *transductive*, and to label a new point you fit again or train a classifier on its output.

6.2 Gaussian mixtures

A Gaussian mixture models the data as coming from k Gaussians with unknown means, covariances and weights, fitted by expectation–maximisation:

$$p(\mathbf{x}) = \sum_{j=1}^k \pi_j \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j).$$

Clusters are ellipsoids, so they may be elongated and of different sizes; assignment is soft, `predict_proba` returning a distribution over components; and — being a density model — it also scores how likely a new point is, which is anomaly detection.

6.3 Anomaly detection, two ways

By density: a Gaussian mixture is a density, so score every image with `gmm.score_samples` and call the lowest few anomalous. By reconstruction error: an image the eigenfaces cannot rebuild, `pca.inverse_transform(pca.transform(X))`, is unlike the images that built them. Plant twelve corrupted images — four rotated, four dimmed, four double-exposed — and count how many each detector puts in its top twelve. Both detectors are fitted on the clean corpus only, so neither has learned the images it is asked to find; the notebook prints how many each catches.

7 Spending the forty labels

Split the corpus 280 / 120, stratified by identity, and label forty of the 280 training photographs. Where they are spent turns out to matter more than what is done with them. Forty at random; forty chosen as the photograph nearest each of forty cluster centroids (`km.transform(Z).argmin(axis=0)`); the representative's label propagated to its whole cluster, 280 labels from 40; propagated only to the 75% of each cluster closest to its centroid; and, as the ceiling the project cannot buy, all 280 true labels. Same classifier, same held-out faces, twenty splits; the only thing that varies is which forty were labelled.

Labels used	Held-out accuracy	sd over 20 splits
40 at random	48.1%	3.8%
40, one per cluster	60.5%	4.8%
Propagated to the whole cluster	55.6%	4.8%
Propagated to the closest 75%	55.1%	4.9%
All 280 true labels — the ceiling	97.4%	1.6%

Twenty splits, because the differences between the middle three rows are of the same order as their spread, and a comparison that turns on less than the spread is not a comparison. *Propagation buys 280 labels from 40 and does worse than the 40 alone*, because a wrong propagated label is worse than no label. Report that; do not bury it.

7.1 The diagnosis was right, and the obvious repair still fails

Representation	Best k	Silhouette	ARI, all 400
PCA to 123 dimensions	60	0.205	0.524
PCA 123, first 3 components discarded	60	0.184	0.508
Unit-norm each image, then PCA 123	58	0.189	0.464

Neither helps. The leading components carry pose and face shape as well as illumination, so removing them throws away signal with the nuisance. The remedy that works is a representation learned *for the task* rather than for variance — which is what Part VI builds, and why a pretrained encoder beats a hand-chosen projection.

Note also what chose k there: sweeping k and keeping the best ARI would make both repairs look like gains, because ARI is computed from the labels this whole lecture is pretending not to have.

8 The notebook

What to change

Change `n_components=0.95` to `0.99`, which takes d from 123 to 260. The sweep slows by roughly the ratio of the two and the ARI barely moves — the whole trade-off in one edit.

9 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 9's deck and in the exercise book.

1. PCA is derived by maximising $\|XW\|_F^2$ subject to $W^T W = I$. State why the data must be centred first, and what the first component becomes if it is not. [5]
2. The explained variance ratio is sometimes called a heuristic for choosing d . Using the result of the PCA derivation, say why it is not one, and what keeping 95% of the variance leaves as reconstruction error. [5]
3. A pipeline fits PCA on the whole dataset and then splits. Name what leaked, and say which held-out number the leak flatters and which it leaves alone. [4]
4. Inertia always falls as k rises, so it cannot choose k . State what silhouette adds, and one situation where it also fails. [5]
5. You have 4,096-dimensional face vectors and are told two faces are “close”. Explain why that alone does not mean they are the same person. [4]

Summary

PCA is the SVD of the centred data: the principal subspace minimises squared reconstruction error exactly, and the error it leaves is $\sum_{j>d} \sigma_j^2$, readable before you choose d . Minimising that error and maximising the projected variance are the same problem by Pythagoras, and the explained variance ratio is that theorem normalised — 123 components hold 95% of 4,096 pixels. A reducer is fitted, so it is fitted after the split: the leak flatters its own objective and leaves the accuracy alone.

Inertia cannot choose k : it is monotone in k and zero at $k = m$. The silhouette can, at $O(m^2 n)$, and must be read against a random anchor — and then against the faces. Compressing to 123 dimensions cuts the factor of n in both cost models by 33.

The lecture opened by measuring that pixel distance is partly identity and partly lighting, and closed by confirming it: mean cluster purity 0.74, the worst cluster a lighting condition rather than a person, and two one-line repairs that both made things worse. DBSCAN and Gaussian mixtures drop assumptions k -means makes; a mixture's density and the PCA reconstruction error both score anomalies. And forty labels spent on cluster representatives were worth a quarter more accuracy than forty spent at random — while propagating them to 280 was worse than not propagating at all.